

Local Resource & Relief Tracker

2-Person Team — Project Idea, Architecture & Development Roadmap

Project Goal: Build a streamlined community platform where people can offer available resources or request resources they need, with simple matching and status tracking.

1. Core Concept

The platform helps communities exchange useful resources during crises or periods of economic strain. A user can post an **offer** (for example, extra food or clothing) or create a **request** (for example, school books or food). Other users can discover resources, respond to requests, and track fulfillment.

2. Core Features — MVP

Feature	Description
Authentication	Register, login, logout, user profiles, and user/admin roles.
Resource Offers	Post food, clothing, medicine, education materials, shelter, transportation, or other resources.
Resource Requests	Request needed resources with quantity, location, urgency, and description.
Browse Resources	Bento-style cards with search and filters for category, location, type, urgency, and status.
Resource Details	View full information and contact/request the resource provider.
User Dashboard	Manage personal offers and requests and see their current status.

3. Key Feature: Resource Matching

A request can be automatically compared against available offers. For example, a request for 10 units of food in Dhanmondi can be matched with available food offers in the same location whose quantity is sufficient. The frontend can then show **“3 matching resources found.”** This gives the project a meaningful application feature beyond basic CRUD.

4. UI Direction

Use a sleek, dark-themed **Bento grid** interface with clean cards, geometric sans-serif typography, strong spacing, and highly readable information. Avoid a cluttered map as the primary interface. The dashboard should emphasize quick scanning, searching, urgent requests, and available resources.

5. Technology Stack

Layer	Technology
Frontend	React + Vite + React Router + Axios + CSS
Backend	Node.js + Express.js + JWT + bcrypt
Database	MySQL
Development	Git + GitHub + feature branches + Pull Requests

6. MySQL Database Design

Keep the relational design simple and normalized enough for a first version.

Table	Important Columns
users	id (PK), name, email, password, phone, location, role, created_at
resources	id (PK), user_id (FK), type, title, description, category, quantity, location, urgency, status, created_at, expires_at
matches	id (PK), resource_id (FK), request_id (FK), provider_id (FK), requester_id (FK), status, created_at

Example resource record

type = offer | title = "20 Meal Boxes" | category = food | quantity = 20 | location = Dhanmondi | status = available

7. REST API Plan

Area	Endpoints
Authentication	POST /api/auth/register POST /api/auth/login GET /api/auth/me
Resources	GET /api/resources GET /api/resources/:id POST /api/resources PUT /api/resources/:id DELETE /api/resources/:id
Offers / Requests	GET /api/resources/offers GET /api/resources/requests
Matching	GET /api/requests/:id/matches POST /api/matches PUT /api/matches/:id

8. Team Division — 2 People

Person 1 — Frontend & UI

React/Vite setup; routing; Navbar; Login/Register; Dashboard; Resource Cards; Search and Filters; Resource Details; Create Offer/Request; User Dashboard; responsive design; API integration.

Person 2 — Backend & Database

Express setup; MySQL connection; database schema; authentication/JWT; User and Resource APIs; CRUD; validation; search/filter APIs; matching algorithm; authorization; admin APIs.

9. Four-Week Development Roadmap

Week 1 — Foundation

Frontend: React/Vite, routing, layout, Navbar, Login/Register, initial Dashboard UI.

Backend: Node/Express, MySQL connection, users table, registration API, login + JWT.

Milestone: React → Express → MySQL connection with working authentication.

Week 2 — Core Resource System

Frontend: Resource Cards, list/details, search, filters, create-resource form.

Backend: resources table, complete CRUD APIs, authentication/authorization checks.

Milestone: Register → Login → Create resource → View → Edit/Delete.

Week 3 — Real Application Features

Frontend: My Dashboard, offers, requests, status/urgency indicators, responsive UI.

Backend: search/filter APIs, offer/request logic, matching algorithm, match APIs.

Milestone: A request can discover suitable available resources.

Week 4 — Testing & Deployment

Both: integration testing, error handling, loading/empty states, mobile responsiveness, security review, UI polish, deployment, documentation, and final demo preparation.

Rule: Avoid major new features during Week 4.

10. Recommended Project Structure

Frontend

```
frontend/src/components/ - Navbar, ResourceCard, SearchBar, Filter
frontend/src/pages/ - Login, Register, Dashboard, Resources, ResourceDetails, CreateResource, Profile
frontend/src/services/ - api.js
frontend/src/App.jsx, main.jsx
```

Backend

```
backend/models/ - User, Resource, Match
backend/routes/ - authRoutes, resourceRoutes, matchRoutes
backend/controllers/ - authController, resourceController, matchController
backend/middleware/ - auth, admin
backend/config/ - db.js
backend/server.js
```

11. GitHub Team Workflow

Use one shared repository and feature branches. Examples: **feature/login**, **feature/dashboard**, **feature/resources-api**, and **feature/matching**. Each person commits and pushes their feature branch, then opens a Pull Request for review before merging into main.

Typical workflow:

```
git checkout -b feature/login
git add .
git commit -m "Add login page"
git push origin feature/login
```

12. Final Feature Set

Priority	Feature
Must Have	Registration/Login, JWT authentication, resource offers, resource requests, search, filtering, details, user dashboard

Core Differentiator	Resource matching system and status tracking
Recommended	Admin panel, responsive UI, GitHub collaboration, deployment
Optional	Map integration, notifications, email notifications, statistics, reputation, image uploads, AI-based matching

13. Development Principle

Do not build the frontend and backend completely separately for three weeks. After each backend API is ready, immediately connect it to the React frontend. This ensures that the team always has a working application and avoids a large integration problem at the end.

Target architecture: React + Vite → Express.js REST API → MySQL